# Week 3 Design Review — Full Presenter Briefing

This document prepares each presenter to deliver their sheets **alone**. Each section gives you: what the sheet shows, the background you need (even if you did not build that part), a suggested script you can nearly read aloud, plain explanations of every technical term on the sheet, the reasons behind the design, and the questions most likely to come at you, with answers.

---

## Part 0 — The story so far (everyone read this first)

**What the project is.** Two vehicles drive in areas with no cell coverage. Each vehicle carries two small devices: a **radio** (a Heltec V4 board running Meshtastic, an open-source long-range radio firmware, with a GPS module attached) and a **display** (a Waveshare board with a 4.2 inch sunlight-readable screen, running firmware we wrote ourselves). The radios exchange encrypted position broadcasts over **LoRa**, a long-range low-power radio technology at 915 MHz. Each display talks to its own radio over Bluetooth and turns the numbers into what a driver actually needs: an arrow pointing at the partner vehicle, the distance, and how fresh that information is.

**What happened in weeks 1 and 2.** We set up the shared repository and toolchain, agreed the interface contract between the two halves of our firmware (a small structure called TrackerState), received and bench-verified all the equipment, flashed the radio, and proved every interface with off-the-shelf tools before writing firmware.

**What happened in week 3 — the reason this review has photos.** The firmware went live. The display now connects to the radio on its own, decodes real position packets, and renders the driver screens. Three solo drive tests were run with one radio (tracking itself — the second radio is not provisioned yet). Each drive test found real problems, and each was fixed the same day:

1. **The frozen location.** The display showed the same coordinates for days. Cause: the radio channel had a privacy default that rounded every broadcast position to a grid about 6 km wide. Fix: full precision enabled.
2. **The wrong speed.** The display showed "72" while driving 45 mph. Cause: the radio transmits speed in km/h and our firmware read it as m/s. Fix: convert at the input; display in mph.
3. **The wrong clock.** One hour off. Cause: our timezone map put Lubbock in Mountain time; West Texas is actually Central. Fix: corrected boundary.
4. **The 30 second lag.** Updates arrived every 30 s even after we sped up broadcasts to 10 s. Cause: the GPS chip itself only computed a new fix every 30 s, so the radio kept re-sending the same old fix. Fix: GPS refresh now matches the broadcast rate at 10 s.
5. **The ghost.** A parked radio with no satellite view kept re-broadcasting its last moving fix, and the display showed "21 mph" as if it were fresh. Fix: the firmware now detects bit-identical re-broadcasts and lets the data age keep counting, so stale data looks stale.

We also survived a real disaster: the radio was accidentally flashed with the wrong firmware and fully recovered in minutes using our own documented recipe. That recipe is now team property.

**The one-sentence story of this review:** *last time we showed drawings; this time we show a running system and the physics that explains it.*

---

## Sheet 01 — Cover (opener, ~1 minute)

**What it shows.** Title, the week-3 claim, the responsibility table, and a drawing-style title block with the numbers that matter.

**Script.** "This is our week 3 design review. Two weeks ago we presented an architecture; this week we present a working tracker. The display now shows a live vehicle: position decoded from real radio packets, an arrow computed from real navigation math, and a screen that moves every second. We are at 29 percent completion against a 19 percent target for week 3, with 60 hours logged. Each of us will walk our own subsystem: first its architecture, then the working hardware and the code behind it."

Then hand off to Deo. Do not explain the table; it explains itself.

---

## Sheets 02–04 — DEO (backend)

### Sheet 02 — The Complete System on One Sheet

**What it shows.** The full system diagram: hardware on the left, our firmware on the right, and a strip at the bottom describing the system the way a telecommunications engineer would.

**Background.** This diagram is the map for the entire presentation. Every later sheet zooms into one region of it.

**Script.** "Before the details, the whole system on one page. Left side: one vehicle's hardware. A GPS module produces a position fix once per second and feeds it over a serial wire to the Heltec radio, which runs Meshtastic, an open-source radio firmware we did not write and did not modify. The radio broadcasts that position through the roof antenna, and receives the partner's broadcasts the same way. A meter away sits our display, connected to the radio over Bluetooth. Both are powered from the vehicle, together, on purpose — you will hear why from Cannon.

Right side: the inside of the display, the only custom firmware in the project. My chain — the protocol side — receives the radio's data stream, decodes it, and writes everything into one shared structure called TrackerState. Josiah's chain — navigation and rendering — reads that structure and draws the screen. One writer, one reader, and they meet at exactly one place.

Bottom strip: the same system as layers. Our application payload is about 40 bytes. The network layer floods packets through any Meshtastic device that hears them, encrypted with AES-256. And the physical layer is the remarkable part: it decodes signals fifteen decibels below the noise floor, which is where the kilometers of range come from. Victor will show you that math.

Last point, and it is the design philosophy: Vehicle B is an identical copy. There is no server and no master. Losing any single device degrades one vehicle's view and nothing else."

**Terms on this sheet, in plain language.** - *Meshtastic:* free, open-source firmware that turns hobbyist radio boards into a self-forming encrypted mesh network. We use it unmodified. - *LoRa:* "long range" radio modulation. Slow on purpose; the slowness buys distance. Victor's sheets explain the trade precisely. - *BLE GATT:* the Bluetooth Low Energy standard for exposing data. The radio offers three "slots": one we write commands to, one we read messages from, and one that notifies us when new messages are waiting. - *TrackerState:* our shared C structure: my fix, partner's fix, link status, and the GPS clock. The single interface between backend and frontend.

**If asked.** - *"Why not write one firmware for everything?"* Everything hard about radio — encryption, retries, mesh relaying, GPS parsing — ships pre-tested inside Meshtastic, maintained by a large open project. We write a client of its documented API. Smallest custom surface, smallest risk. - *"What if you lose the Bluetooth link?"* The display keeps rendering the last known data with an honest age, and reconnects automatically. It never goes blank.

## Sheet 03 — Backend: the Data Spine

**What it shows.** The firmware architecture diagram, with my three modules on the left: ble_link, proto_decode, node_table.

**Background you need.** The trickiest part of this project was not decoding data; it was *connecting* reliably. Two discoveries shaped the design. First, a Meshtastic radio only advertises its Bluetooth presence in a short window after it boots; after the first disconnect it goes quiet, and scanning for it will never find it again. Second, if the display stores a Bluetooth bond the radio stops advertising entirely. Both discoveries cost us bench days and both are designed around, permanently.

**Script.** "My chain runs top to bottom. ble_link is a Bluetooth central — a client. It does not scan first; it connects directly to the radio's address, which it remembers in flash storage. That is deliberate: this radio only advertises briefly at boot, so a direct connection during that window is the only reliable way in. That is also why both units power on together with the ignition. The link encrypts using the simplest pairing mode, subscribes to a notification that acts like a doorbell — 'new data waiting' — and then reads the radio's message queue until it is empty.

Each message goes to proto_decode. The radio speaks protobuf, a compact binary format defined by a published schema. We do not parse bytes by hand: a tool called nanopb generates the decoder from Meshtastic's own schema files, so our decoder cannot disagree with the radio's encoder.

Decoded positions land in node_table, which keeps the latest fix for our vehicle and the partner's, each with an honest timestamp, and once per second publishes the whole picture into TrackerState. That structure is version 2 now: it carries speed, data ages, and the GPS wall clock that drives the time display.

Status: this chain is not bench-proven, it is road-proven. Across four drive tests the link dropped and reconnected on its own every time, positions decode at the ten second broadcast rate, and a radio that loses GPS lock can no longer pass off old data as fresh."

**Terms.** - *Central/peripheral:* Bluetooth roles. The display is the central (client); the radio is the peripheral (server). - *protobuf:* a message format with two parts — a schema (a contract both sides compile from) and a very compact binary encoding. A GPS fix costs about 25 bytes instead of about 90 as text. - *nanopb:* a tiny C implementation of protobuf for microcontrollers; generates our decoder from the schema. - *NVS:* non-volatile storage, a small key-value store in flash that survives power-off. Holds the radio's remembered address.

**If asked.** - *"What is the handshake?"* On connect we send one command, want_config, with a random number. The radio replays its identity and its whole node database, then echoes our number back so we know the snapshot is complete. The display never starts blind. - *"Why no stored bond?"* Bench-proven trap: a bonded Meshtastic radio stops advertising for everyone. We pair fresh on every connection — still encrypted, takes milliseconds.

## Sheet 04 — How the Pieces Connect in Code

**What it shows.** Four real code fragments from four real files, arranged as the actual data path, with the mutex boundary marked.

**Background.** This sheet exists to prove the architecture is not slideware. Every fragment is copy-pasted from the repository.

**Script.** "This is the journey of one packet through real code. Top left, ble_link: a loop that reads the radio's queue until a zero-length read says empty. One doorbell notification can mean five queued messages, so we always drain fully. Top right, proto_decode: one call turns the bytes into named fields — latitude, longitude, speed, GPS time — and hands them to the node table. Bottom left, node_table publishes TrackerState once per second. Note the comment: the ONE writer. Bottom right is Josiah's side, the ONE reader: it copies a snapshot, projects it forward — dead reckoning, he will explain — and computes the arrow.

The arrow between the bottom boxes is the entire team interface. The store functions take a mutex — a lock — internally. Neither of us can forget the lock, because the locked functions are the only door.

The gray card is this week's field hardening, and every line of it came from a real drive test: the radio reports speed in kilometers per hour and we now convert at the door; GPS jitter below walking pace reads as zero so a parked car shows zero; a radio with no satellite lock re-broadcasts its old fix bit-for-bit, and we now detect that so the data age keeps climbing instead of resetting; and the clock is guarded so it can never run backwards."

**Terms.** - *Mutex:* mutual exclusion lock. Only one task may touch the shared struct at a time. Without it, the reader could catch the struct half-updated and display a position that never existed (a "torn read"). - *GATT queue / FromRadio:* the radio's outbox. Every read hands over the next message; empty read means done. - *Deadband:* a threshold below which a noisy measurement is treated as zero.

**If asked.** - *"Why a struct and not a message queue?"* Navigation wants the latest state, not history. A newer fix makes the old one worthless. One always-current snapshot cannot overflow and cannot fall behind. - *"How do you test this?"* The nav math runs on a laptop with no hardware (12 of 12 tests pass), and a simulator can replace my whole chain — the frontend cannot tell the difference. That is the proof the boundary is real.

---

# Sheets 05–06 — JOSIAH (frontend)

## Sheet 05 — Frontend: From Struct to Arrow

**What it shows.** Your pipeline (snapshot → dead reckon → navigate → render) and the two code fragments that do the real work.

**Background you need.** Everything you render comes from one structure, TrackerState, that Deo's side fills once per second: our fix, the partner's fix (each: latitude, longitude, course, speed, age, valid flag), a link flag, and the GPS clock. You read a private copy of it twice a second. You never call into the backend and it never calls into you.

Also know where your inputs come from physically: course and speed are computed by the GPS from its own motion (called course over ground), not from a compass. There is no compass in this system.

**Script.** "My side turns that shared structure into the driver's screen, in four steps, twice a second. First I copy a snapshot under the lock, so everything in one frame is from the same instant. Second, dead reckoning: the radio only measures every ten seconds, but a vehicle cannot teleport, so I project both vehicles forward along their last course at their last speed, one second at a time. The code is three lines: distance traveled equals speed times age; move that far along the course heading. Every real fix snaps the estimate back to truth, so the error between fixes is a few meters of steady driving. I stop projecting past 120 seconds so a ghost freezes instead of sailing off the map. Third, navigation: distance by the haversine formula — the great-circle distance between two points on a sphere — the bearing from us to the partner, and then the arrow is simply bearing minus our own course, wrapped to plus or minus 180. Zero means dead ahead. Fourth, rendering at two hertz: the compass page while driving, a status page with raw coordinates on a key press, hold to reset tracking.

Two design decisions worth defending. We use GPS course over ground instead of a magnetometer because a compass inside a steel cab is unreliable, and a driving use case is usually moving; below five miles per hour the arrow becomes a north-up compass rose, because course is noise when parked. And every element on screen is a state machine on the data: the CHANGE COURSE banner engages after five seconds off course and clears on recovery, the chevrons show which way to turn, the STALE banner appears when data is older than two minutes, and the clock even derives its timezone from position.

The engineering point: both files are pure C with no hardware dependencies. Twelve of twelve unit tests run on a laptop. The whole interface was built and demonstrated before the radio link existed, using simulated data."

**Terms.** - *Dead reckoning:* estimating current position by projecting the last known position forward along course and speed. Ships and aircraft displays have done it for a century; every chartplotter animates between beacon reports. - *Haversine formula:* the standard formula for distance between two latitude/longitude points along the Earth's surface. - *Bearing:* the compass direction from you to the target, measured from north. *Course over ground (COG):* the direction you are actually moving, measured by GPS from your motion. - *Arrow angle = bearing − course:* if the partner is at bearing 90 (due east) and you are driving at course 350, the arrow shows +100: turn right. - *2 Hz:* twice per second. Fast enough for a smooth arrow, well within the reflective panel's redraw budget.

**If asked.** - *"How accurate is dead reckoning?"* Between two fixes the error is bounded by how much the vehicle changes speed or direction in ten seconds; on a road, typically a few meters. Hard turns lag by up to one correction cycle. - *"Why 120 second cap?"* Beyond the stale threshold the projection is fiction. Freezing is honest; sailing away is not. - *"What if the driver is stopped facing away?"* Stopped mode is north-up with a rose; the driver reads it like a paper map. The moment the vehicle moves above five miles per hour it flips to vehicle-relative.

### Sheet 06 — The UI, Running on Real Packets

**What it shows.** Four photographs of the actual display, all taken while the system ran on live decoded radio packets.

**Script.** "These are photographs, not mockups, and there is no simulator behind them: every number arrived by radio and was decoded by our stack. Top left is the driving view: partner 2.41 miles away, data fifteen seconds old, and the state machine is alive — I am off course, so the banner says CHANGE COURSE and the chevron points the way back. Top middle: I stopped, so the display noticed my course was no longer meaningful and fell back to a north-up compass rose on its own. Below it, close range: two tenths of a mile, the arrow oversized for a glance. Bottom middle is the status page, one key press away: raw coordinates for me and the partner, live, which is our debugging view.

One hardware note: this panel is reflective. It gets sharper in more sunlight, the exact opposite of a phone. It was chosen for a dashboard in the sun.

And the honest-data principle: the age readout is on every screen. The driver always knows how fresh the information is, and if the partner's radio goes silent, the age climbs and the display says STALE. This system never pretends."

**If asked.** - *"Why black and white?"* One-bit reflective panel: near-zero power, sunlight readable, fast enough for a live arrow. Color adds nothing for an arrow and a number. - *"Night?"* Ambient light is required; an edge light is on the plan.

---

## Sheets 07–08 — VICTOR (radio and physics)

### Sheet 07 — Radio and PHY: the Long-Range Link

**What it shows.** The radio-node hardware diagram, a photo of the rig, the radio's own status screen, the configuration, and the provisioning warning.

**Background you need.** The radio is a Heltec V4 board running stock Meshtastic. GPS plugs in by a small connector and feeds it text sentences (a format called NMEA) once per second. The radio parses that once and from then on everything is compact binary. All configuration was set over USB using the Meshtastic command line and is documented in hardware/README.md in the repository — that file is your bible for radio number two.

**Script.** "The radio node: GPS in by serial, the LoRa transceiver out to a roof antenna, and a small OLED screen that turns out to be a free field debugger — it shows satellites in view, nodes heard, channel utilization, and the signal strength of the last packet. In the photo it shows nine satellites.

Configuration, tuned by three road tests. We run the LONG_FAST preset: spreading factor 11 across 250 kilohertz of bandwidth with forward error correction, in the 915 megahertz unlicensed band. The channel is private and encrypted with AES-256, at full position precision — that matters because the privacy default rounds positions to a six kilometer grid, which froze our display for two days before we found it. Broadcasts are smart: every ten seconds or twenty meters while moving, every sixty seconds parked. And the GPS refresh is also ten seconds, because we learned the hard way that a ten second broadcast of a thirty second fix just repeats stale data.

The amber card is provisioning discipline for radio number two. It must join with the exact channel URL, because the URL embeds the encryption key — without it the radios cannot decrypt each other. And two traps we documented from experience: some settings silently revert when written over USB, because closing the serial port resets the chip mid-save, so every setting gets read back and verified; and our two boards are identical to a computer, so flashing always names its target port explicitly."

**Terms.** - *Preset / spreading factor / bandwidth:* LoRa's knobs. Spreading factor 11 means each data symbol is stretched into a 2048-step frequency sweep — slow, but detectable far below the noise. Sheet 08 does the numbers. - *ISM band:* license-free radio spectrum (915 MHz in the US), shared by everyone, which is why polite airtime use matters. - *Channel URL:* Meshtastic packs the channel name, settings, and encryption key into one URL. Scan it or paste it and a radio joins the channel. - *ChUtil:* channel utilization, the percent of airtime the radio hears being used. Ours idles near zero; the cap is about 25 percent.

**If asked.** - *"What range do you expect?"* The link budget supports multiple kilometers with the roof antenna; the range campaign in weeks 4 and 5 will measure it on terrain. (Do not promise a number before measuring.) - *"Why is the antenna on the roof?"* At 915 MHz the vehicle body blocks and reflects; getting the antenna outside the metal box is the single biggest range factor, bigger than any setting.

## Sheet 08 — Real Time versus 10 Seconds: the Physics

**What it shows.** Five typeset equations that together answer the most common question we get: why does the position update every ten seconds instead of continuously?

**Background you need.** Read this twice; it is the most technical sheet in the deck, and it is yours. The logic chain is: packets are slow because we chose range; slow packets mean the channel fits few of them; therefore measurements are sparse; but delivery is fast, and projection fills the gaps.

**Script — walk the equations in order.**

"Equation one, airtime. In LoRa, the time per symbol is two to the spreading factor divided by the bandwidth. At spreading factor 11 over 250 kilohertz, that is 8.19 milliseconds per symbol — and a full position packet, with preamble and error correction, occupies the air for about half a second. Half a second, for forty bytes. That is the price of range.

Equation two, utilization. Two vehicles broadcasting once per second at half a second each would demand one hundred percent of the shared channel — collapse; nothing gets through. At a ten second interval they use ten percent, comfortably under the twenty-five percent limit the firmware enforces. So ten seconds is not a choice we made; it is what the channel can afford.

Equation three, sensitivity — the reason we pay that price. Receiver sensitivity is thermal noise, minus 174 dBm per hertz, plus ten log of the bandwidth, plus the receiver's noise figure, plus the minimum signal-to-noise ratio. At spreading factor 11 that minimum SNR is *minus* 17.5 dB — the signal can be seventeen and a half decibels quieter than the noise and still decode, because the receiver knows the chirp pattern to correlate against. Sensitivity comes out near minus 131 dBm. That processing gain is the entire range budget: kilometers on under a watt.

Equation four, the real-time question. Information latency — from a fix existing to the driver seeing it — is airtime plus Bluetooth plus rendering: about one second. The system IS real time in delay. Ten seconds is the sampling period, a different quantity.

Equation five, what fills the gap. The display projects each vehicle forward from its last fix along its course at its speed, once per second, capped at the two minute stale horizon. Measurements every ten seconds, motion every second, and every real fix snaps the estimate back to truth.

The verdict: rate and range are the same currency, and we spent it on range. For precedent: marine AIS transponders beacon every two to thirty seconds and every chartplotter on earth animates between reports. We are in good company."

**If asked.** - *"Could you update faster?"* Yes, by dropping the spreading factor — SF7 packets cost about 40 milliseconds — but that surrenders 12 to 16 dB of link budget, collapsing kilometers to hundreds of meters. It would delete the project's purpose. It would, however, make a great range-versus-rate demo in a parking lot. - *"Where does −17.5 dB come from?"* It is the demodulation SNR limit for SF11, from the LoRa design equations; each doubling of the spreading factor buys roughly 2.5 to 3 dB. The receiver correlates a known 2048-chip pattern, so noise averages out while the signal adds up. - *"Is 10% utilization safe?"* Yes — it leaves room for a third node, for retransmissions, and for the mesh relaying Cannon's tests will exercise.

---

## Sheet 09 — CANNON (integration constraints)

**What it shows.** The system context diagram, the integration test sequence, and six constraint cards, one per subsystem.

**Background you need.** Your role in this review is to show that integration is planned around *measured reality*, not hope. Every card on this sheet is a rule the hardware taught us — you can present each one as a short story of "what happened, and what we do about it." The stories are in Part 0 of this document; the one-line versions are on the cards.

**Script.** "My sheet is the ledger of constraints, and the claim I want to make is that none of them are theoretical. Every card came from our bench or our road tests, and each one becomes a line item in the integration test protocol.

Radio: each packet costs half a second of air, which caps the update rate at ten seconds — Victor just showed you why — and if the region is not set the radio will not transmit at all. The roof antenna is the biggest single range factor.

GPS: it needs sky. A windshield works; a desk does not. Cold start takes minutes, so the test protocol allows warm-up time. Indoors, reflected signals fake one to three kilometers per hour of phantom speed — we saw it, and the firmware deadband removes it. And the fix rate must match the broadcast rate, or the radio re-sends stale fixes; that was a real bug.

Bluetooth: the radio only accepts a connection in a window after boot, so both units power up together — the ignition-on rule. And exactly one client at a time: if someone opens the phone app or a USB session, they steal or mute the display's link. That goes in the field checklist in bold.

Display: one-bit reflective panel, no grayscale, needs ambient light at night; edge lighting is planned.

Power: cheap 12 volt USB adapters shut off on the radio's low current draw — we validate every adapter before it goes in a vehicle.

Provisioning: the channel URL carrying the encryption key must match on every radio; node identities derive from the hardware address and survive reflashing — our recovery drill proved that; and every configuration write gets read back, because we caught writes silently reverting.

The integration sequence: provision radio two from the documented recipe, prove two-radio decode on the bench, then the first two-vehicle drive test, then range characterization on terrain."

**If asked.** - *"What is the riskiest remaining item?"* Two-radio behavior in motion — everything so far is one radio tracking itself. That is exactly what the next test exists to measure. - *"How do you log results?"* The logging harness (my week 4 build) captures RSSI, SNR, and packet delivery per broadcast; drive tests get a dated log in the repository, same as the soak test already there.

---

## Sheet 10 — Status and Next (closer, ~1 minute)

**Script.** "Where we stand: 29 percent complete against a 19 percent target; 60 hours logged; 12 of 12 unit tests passing; measurements every ten seconds with on-screen motion every second. Proven end to end this week: a GPS fix travels radio to radio to display to arrow; the clock sets its own timezone from position; data ages stay honest; the link healed itself after every drop; and we executed a complete radio recovery drill from our own documentation in minutes.

Next: Victor provisions radio number two from the recipe; Cannon turns the constraints ledger into the drive-test protocol; Deo and Josiah add partner selection and night readability. Then the milestone that matters:"

Read the closing bar verbatim: **"One radio still talks to itself. But every layer between a GPS fix and the driver's arrow is now real, tested code. Radio number two turns this into a tracker."**

---

## Appendix — Whole-team Q&A bank

- **Why not stream positions continuously?** Sheet 08, equations one and two: half a second of airtime per packet; two vehicles at one hertz exceed the whole channel. Dead reckoning delivers the continuous feel at zero airtime cost.
- **What happens when a packet is lost?** Nothing breaks. A newer position supersedes an older one, the age counts up, the next broadcast heals it. We display reliability (age) instead of pretending with acknowledgments.
- **How is it secured?** AES-256 on a private channel; the key travels only inside the channel URL, which stays within the team.
- **What was the hardest bug?** The ghost: a radio without GPS lock re-broadcasts its last fix bit-for-bit, and our display stamped it fresh. Fixed by detecting identical re-broadcasts, so age reflects measurement time, not arrival time.
- **How do you know your math is right?** Twelve native unit tests on every build; the physics on sheet 08 matches measured behavior (about half a second of airtime, utilization visible on the radio's own screen); and the displayed positions were verified against the Meshtastic map.
- **Can it scale to three vehicles?** Yes. A third radio just joins the channel; the node table already tracks multiple nodes; "select partner" becomes "cycle partners." No redesign.